University of Science

Faculty of Information Technology

Data Structures and Algorithms Project Report

Topic: SORTING ALGORITHMS

Course: Data Structures & Algorithms

Instructor:
Trần Thị Thảo Nhi
Trần Hoàng Quân

Class: 25C10
Group ID: 07

Group Members:
Đặng Bá Lâm - 25127403
Hứa Minh Khang - 25127073
Trịnh Bảo Khôi - 25127256
Nguyễn Hữu Ngọc Thuận - 25127153

Ho Chi Minh City, March 13, 2026

Mục lục

1 Group Information **3**

2 Algorithms Presentation **4**
2.1 Selection Sort . . . 4
2.1.1 Overview . . . 4
2.1.2 Idea . . . 4
2.1.3 Step-by-Step Algorithm . . . 4
2.1.4 Pseudo Code . . . 4
2.1.5 Complexity . . . 5
2.1.6 Variants and Optimizations . . . 5
2.2 Insertion Sort . . . 6
2.2.1 Overview . . . 6
2.2.2 Idea . . . 6
2.2.3 Step-by-Step Algorithm . . . 6
2.2.4 Pseudo Code . . . 6
2.2.5 Complexity . . . 7
2.2.6 Variants and Optimizations . . . 7
2.3 Shell Sort . . . 8
2.4 Bubble Sort . . . 9
2.5 Heap Sort . . . 10
2.6 Merge Sort . . . 11
2.6.1 Overview . . . 11
2.6.2 Idea . . . 11
2.6.3 Step-by-Step Algorithm . . . 11
2.6.4 Pseudo Code . . . 11
2.6.5 Complexity . . . 12
2.6.6 Variants and Optimizations . . . 12
2.7 Quick Sort . . . 13
2.7.1 Overview . . . 13
2.7.2 Idea . . . 13
2.7.3 Step-by-Step Algorithm . . . 13
2.7.4 Pseudo Code . . . 13
2.7.5 Complexity . . . 14
2.7.6 Variants and Optimizations . . . 14
2.8 Radix Sort . . . 15
2.8.1 Overview . . . 15
2.8.2 Idea . . . 15
2.8.3 Step-by-Step Algorithm . . . 15
2.8.4 Pseudo Code . . . 15
2.8.5 Complexity . . . 16
2.8.6 Variants and Optimizations . . . 16
2.9 Binary Insertion Sort . . . 17
2.10 Shaker Sort . . . 18
2.11 Flash Sort . . . 19

3 Experimental results and comments **20**

4 Project Organization and Programming Notes **21**

5 Acknowledgements **22**

A Tài liệu tham khảo **23**

1 Group Information

Student Name	ID	Primary Responsibilities	Contribution %
Đặng Bá Lâm	25127403	[X]%	
Hứa Minh Khang	25127073	[X]%	
Trịnh Bảo Khôi	25127256	[X]%	
Nguyễn Hữu Ngọc Thuận	25127153	[X]%	
Total			**100%**

**Note: All members participated in Time Calculator implementation, Report writing, Citations, and Final checking.*

2 Algorithms Presentation

2.1 Selection Sort [2]

2.1.1 Overview

- **History and Origin:** A simple and intuitive in-place comparison-based sorting algorithm.
- **Efficiency:** Generally performs worse than similar algorithms like Insertion Sort. Best suited for situations where memory writes are costly.
- **Application:** Useful for small datasets or when auxiliary memory is extremely limited.

2.1.2 Idea

- The algorithm divides the input list into two parts: a sorted sublist built up from left to right at the front, and a sublist of the remaining unsorted items.
- Initially, the sorted sublist is empty and the unsorted sublist is the entire list.
- It proceeds by finding the smallest element in the unsorted sublist, swapping it with the leftmost unsorted element, and moving the sublist boundary one element to the right.

2.1.3 Step-by-Step Algorithm

- Consider an array of n elements, a[0. . . (n - 1)].
- **Step 1:** Start a loop with variable i from 0 to n - 2.
- **Step 2:** Assume the element at i is the minimum ($min = i$).
- **Step 3:** Iterate through the remaining array from $j = i + 1$ to n - 1 to find the actual minimum element.
  - If a[j] $<$ a[min], update $min = j$.
- **Step 4:** Swap a[i] with a[min].
- **Step 5:** Increase i by 1 and repeat until the array is fully sorted.

2.1.4 Pseudo Code

- **Algorithm:** Selection Sort
- **Input:** An array A of n elements

```
i = 0
While i < n - 1:
   min = i
   For j = i + 1 to n - 1:
      If A[min] > A[j]:
```

```
        min = j
    Swap(A[i], A[min])
    i = i + 1
```

2.1.5 Complexity

- **Time Complexity:**
  - Best case: $O(n^2)$
  - Average case: $O(n^2)$
  - Worst case: $O(n^2)$

    The algorithm scans the entire remaining array regardless of the initial order.
- **Space Complexity:** $O(1)$

  In-place algorithm, requiring only a constant amount of extra memory for swapping.

2.1.6 Variants and Optimizations

- **Current Implementation (Standard):**
  - The provided C++ code implements the standard unidirectional Selection Sort, which scans for the minimum element from left to right in a single pass.
- **Theoretical Variant: Bidirectional Selection Sort [2]:**
  - **Problem:** Conventional Selection Sort only finds the minimum element in each pass, leaving the rest of the array unsorted.
  - **Solution:** In each pass, simultaneously find both the minimum and maximum elements, placing them at the beginning and end of the unsorted section respectively.
  - **Benefit:** Reduces the total number of iterations by half compared to the implemented version, improving practical execution time.

2.2 Insertion Sort [1]

2.2.1 Overview

- **History and Origin:** One of the oldest and most intuitive sorting algorithms. Mimicking how humans sort a hand of playing cards.
- **Efficiency:** Best for small datasets or nearly sorted/sorted data. Worst for reverse-sorted datasets.
- **Application:** Often used as the base case for hybrid algorithms by Tim Sort, Intro Sort, Quick Sort, etc. to handle small sub-lists.

2.2.2 Idea

- Let the list be divided by sub-lists, separating them into sorted and unsorted sections.
- Take the first element of the unsorted list and insert it into its proper place in the sorted one.
- After each placement, the size of the sorted region grows by 1 and the size of the unsorted region shrinks by 1.

2.2.3 Step-by-Step Algorithm

- Consider an array of n elements, a[0...(n - 1)].
- **Step 1:** Start an increase variable i that is from 1 to n - 1.
- **Step 2:** Find the correct position *pos* in a[0...(i - 1)] to insert a[i], where a[pos - 1] $<$ a[i] $<$ a[pos].
  - Set val = a[i], then move forward a[pos...(i - 1)] to one element. After that, set a[pos] = val.
  - Increase i by 1.
- **Step 3:** If i $<$ n return to **Step 2**. Otherwise, stop the algorithm.

2.2.4 Pseudo Code

- **Algorithm:** Insertion Sort
- **Input:** An array A of n elements

```
For i = 1 to n - 1:
   val = A[i]
   j = i - 1
   While j >= 0 and A[j] > key:
      A[j + 1] = A[j]
      j = j - 1
   A[j + 1] = key
```

2.2.5 Complexity

- **Time Complexity:**
  - Best case: $O(n)$
    When the input is already sorted.
  - Average case: $O(n^2)$
    When the input is random.
  - Worst case: $O(n^2)$
    When the input is reverse-sorted.
- **Space Complexity:** O(1)
  In-place algorithm. Requires only a constant amount of extra memory.

2.2.6 Variants and Optimizations

- **The "Hit" Method (Enhanced Insertion Sort)[6]:**
  - **Problem:** Conventional Insertion Sort is inefficient for reverse-sorted lists due to maximum comparisons.
  - **Solution:** Immediately compare $A[i]$ with $A[0]$ (the current minimum).
  - **Benefit:** If $A[i] < A[0]$, it is moved to the front instantly, bypassing all intermediate comparisons and significantly reducing execution time.
- **Binary Insertion Sort:**
  - Use Binary Search to find the position of an element within the sorted sub-array which is greater than or equal to A[i].
  - While Binary Search reduces the number of comparisons to $O(n \log n)$, the complexity of data movement remains $O(n^2)$ because elements must still be shifted to make space for the new entry.

2.3 Shell Sort

2.4 Bubble Sort

2.5 Heap Sort

2.6 Merge Sort [3]

2.6.1 Overview

- **History and Origin:** Merge Sort is a divide-and-conquer algorithm that was invented by John von Neumann in 1945. Merge sorting is also known as a divide and conquer method of sorting features, and it is based on this method (Varshney & Yadav, 2014).
- **Efficiency:** Consistently performs well on large datasets.
- **Application:** Commonly used in external sorting and when stable sorting is required.

2.6.2 Idea

- The array is recursively divided into two halves until each sub-array contains a single element.
- The sub-arrays are then merged back together in a sorted manner.

2.6.3 Step-by-Step Algorithm

- Consider an array of n elements, a[0. . . (n - 1)].
- **Step 1:** Recursively divide the array into halves until it reaches sub-arrays of size 1.
- **Step 2:** Merge the sorted sub-arrays back together by using two pointers to track the current position in the left and right sub-arrays, respectively. If there are remaining elements in the left sub-array, copy them to a temporary array. Repeat the same for the right sub-array.
- **Step 3:** Copy the sorted temporary array back to the original array a[left. . . right].
- **Step 4:** Repeat the process until the entire array is merged and the recursion stack is empty.

2.6.4 Pseudo Code

- **Algorithm:** Merge Sort
- **Input:** An array A of n elements

```
MergeSort(arr[], l, r):
    If l < r:
        middle index: m = l + (r - l)/2
        Call mergeSort(arr, l, m)
        Call mergeSort(arr, m + 1, r)
        Call merge(arr[], l, m, r) to merge the two halves

merge(arr[], l, m, r):
    Set pos = left, leftPos = left, rightPos = mid + 1
    While leftPos <= mid and rightPos <= right:
        tempArray[pos++] = min(a[leftPos], a[rightPos])
        If a[leftPos] <= a[rightPos]: increment leftPos
        else: increment rightPos
```

2.6.5 Complexity

- **Time Complexity:**
  - Best case: $O(n \log n)$
  - Average case: $O(n \log n)$
  - Worst case: $O(n \log n)$
- **Space Complexity:** O(n)

  Requires additional memory for the temporary array used in the merge process.

2.6.6 Variants and Optimizations

- **The Hybrid Sort (Mixing between Quick Sort and Merge Sort)**
  - **Step 1:** Split the array [a] into [n] parts.
  - **Step 2:** Select part [1]
  - **Step 3:** Sort it using Quick Sort.
  - **Step 4:** Select part [2] and implement the same process until part [n] is sorted.
  - **Step 5:** Merge the sorted parts using Merge Sort.

2.7 Quick Sort [4]

2.7.1 Overview

- **History and Origin:** Developed by Tony Hoare in 1959. One of the most famous divide-and-conquer algorithms.
- **Efficiency:** Highly efficient and generally outperforms other $O(n \log n)$ algorithms in practice due to excellent cache locality.
- **Application:** The default sorting algorithm in many standard libraries (e.g., C++ `std::sort`).

2.7.2 Idea

- It works by selecting a 'pivot' element from the array and partitioning the other elements into two sub-arrays according to whether they are less than or greater than the pivot.
- The sub-arrays are then sorted recursively.
- The base cases are empty arrays or arrays of a small size, where a simpler algorithm (like Insertion Sort) is often used to finish sorting.

2.7.3 Step-by-Step Algorithm

- **Step 1:** If the array segment has less than 10 elements, apply **Insertion Sort** to quickly sort it and terminate the recursion.
- **Step 2:** Select a pivot using the **Median-of-Three** method (finding the median of the first, middle, and last elements).
- **Step 3:** Swap the chosen pivot with the second-to-last element to temporarily move it out of the way.
- **Step 4:** Use two pointers, *left* and *right*, starting from the ends of the segment.
  - Move *left* forward until finding an element larger than the pivot.
  - Move *right* backward until finding an element smaller than the pivot.
  - Swap these two elements if $left \leq right$.
- **Step 5:** Restore the pivot to its correct final position.
- **Step 6:** Recursively apply Quick Sort to the sub-array before the pivot and the sub-array after the pivot.

2.7.4 Pseudo Code

- **Algorithm:** Quick Sort (Optimized)
- **Input:** An array A, left index l, right index r

```
1. If (r - l) < 10:
2.    InsertionSort(A, l, r)
3.    Return
4. p = Partition(A, l, r) // Uses Median-of-Three
5. QuickSort(A, l, p - 1)
6. QuickSort(A, p + 1, r)
```

2.7.5 Complexity

- **Time Complexity:**
  - Best case: $O(n \log n)$
  - Average case: $O(n \log n)$
  - Worst case: $O(n^2)$

    Occurs when the partition is highly unbalanced, though the Median-of-Three optimization makes this extremely rare.
- **Space Complexity:** $O(\log n)$

  Bounded by the call stack depth during recursion.

2.7.6 Variants and Optimizations

- **Implemented Optimization 1: Median-of-Three Pivot Selection**
  - **Implementation:** The `selectPivot` function in the provided C++ code chooses the median of the first, middle, and last elements ($A[l], A[mid], A[r]$) to use as the pivot.
  - **Benefit:** Avoids the $O(n^2)$ worst-case performance on already sorted or reverse-sorted arrays, ensuring a more balanced partition in practical scenarios.
- **Implemented Optimization 2: Hybrid Sorting (Insertion Sort Threshold)**
  - **Implementation:** The `quickSortArray` function checks if the sub-array size is less than 10 (`(r - l) < 10`). If so, it halts recursion and delegates the sorting to `insertionSortSubmatrix`.
  - **Benefit:** Eliminates the overhead of recursive function calls for very small arrays, where simple $O(n^2)$ algorithms like Insertion Sort execute much faster due to lower constant factors.

2.8 Radix Sort [5]

2.8.1 Overview

- **History and Origin:** A non-comparative sorting algorithm originally used for sorting punch cards.
- **Efficiency:** Highly efficient for datasets with keys that are not too large (e.g., integers or strings).
- **Application:** Widely used in environments where numerical data needs to be sorted in linear time relative to the number of digits.

2.8.2 Idea

- Radix Sort avoids direct comparison between elements. Instead, it groups elements by their individual digits.
- The implemented version is Least Significant Digit (LSD) Radix Sort, which processes digits from the least significant (rightmost) to the most significant (leftmost).
- It relies on a stable sorting subroutine (like Counting Sort) to sort the digits at each positional place value.

2.8.3 Step-by-Step Algorithm

- Consider an array of n elements.
- **Step 1:** Find the maximum value ($maxv$) in the array to determine the number of digits.
- **Step 2:** Initialize a multiplier $j = 1$.
- **Step 3:** While $maxv$ / $j > 0$, perform Counting Sort for the current digit place:
  - Create a frequency array *Count* of size 10 (for base 10) initialized to 0.
  - Calculate occurrences of each digit: *digit* = (array[i] / j) % 10.
  - Accumulate *Count* array to determine actual positions.
  - Place elements into a temporary array B traversing backwards to maintain stability.
  - Copy array B back to the original array.
- **Step 4:** Multiply j by 10 and return to **Step 3** until all digits are processed.

2.8.4 Pseudo Code

- **Algorithm:** LSD Radix Sort
- **Input:** An array A of n elements

```
Find maxv in A
j = 1
While (maxv / j) > 0:
   Count = array of 10 zeros
   For i = 0 to n - 1:
      digit = (A[i] / j) % 10
      Count[digit]++
   For k = 1 to 9:
      Count[k] = Count[k] + Count[k - 1]
   B = array of size n
   For i = n - 1 down to 0:
      digit = (A[i] / j) % 10
      B[Count[digit] - 1] = A[i]
      Count[digit]--
   Copy B back to A
   j = j * 10
```

2.8.5 Complexity

- **Time Complexity:**
  - Best, Average, and Worst case: $O(d(n+k))$

    Where d is the number of digits, n is the number of elements, and k is the base (10).
- **Space Complexity:** $O(n+k)$

  Requires auxiliary arrays (*Count* and B) for the internal Counting Sort.

2.8.6 Variants and Optimizations

- **Current Implementation (Base-10 LSD):**
  - The provided C++ code implements Least Significant Digit (LSD) Radix Sort using base 10 (decimal system). It extracts digits using modulo (`% 10`) and allocates a counting array of size 10 for each pass.
- **Theoretical Variant: Bitwise Radix Sort (Base-256):**
  - **Problem:** Using base 10 requires division (`/`) and modulo (`%`) operations, which are computationally expensive. Additionally, base 10 requires more passes for large integer limits.
  - **Solution:** Change the radix base to a power of 2 (e.g., base 256 for byte-level extraction) using fast bitwise operations (right shift `»` and bitwise AND `&`).
  - **Benefit:** Drastically reduces the number of sorting passes (e.g., only 4 passes for 32-bit integers) and replaces slow arithmetic operations with fast bitwise logic.

2.9 Binary Insertion Sort

2.10 Shaker Sort

2.11 Flash Sort

3 Experimental results and comments

4 Project Organization and Programming Notes

5 Acknowledgements

A Tài liệu tham khảo

Tài liệu

[1] Humaira Ali, Haque Nawaz, and Abdullah Maitlo. Performance analysis of heap sort and insertion sort algorithm. *International Journal*, 9(5), 2021.

[2] Jehad Alnihoud and Rami Mansi. An enhancement of major sorting algorithms. *The International Arab Journal of Information Technology*, 7(1):55–58, 2010.

[3] Ghaniyyat Bolanle Balogun, Muhideen Abdulraheem, Peter Ogirima Sadiku, Olawale Debo Taofeek, and Adebisi Sodiq Adewale. Halstead's complexity measure of a merge sort and modified merge sort algorithms. *Jambura Journal of Informatics*, 5(2):141–151, 2023.

[4] Markus E Nebel and Sebastian Wild. Pivot sampling in dual-pivot quicksort. In *25th International Conference on Probabilistic, Combinatorial and Asymptotic Methods for the Analysis of Algorithms*, pages 325–338, 2014.

[5] Julian Shun. Theoretically-efficient and practical parallel in-place radix sorting. *arXiv preprint arXiv:1906.09697*, 2019.

[6] Tarundeep Singh Sodhi, Surmeet Kaur, and Snehdeep Kaur. Enhanced insertion sort algorithm. *International journal of Computer applications*, 64(21):35–39, 2013.